CS 584 Natural Language Processing

Lecture 9: Reasoning Agents

Yue Ning
Department of Computer Science
Stevens Institute of Technology

Today's Lecture

- Reasoning
- Language Model Agent
- Practical considerations and safety concerns

Reasoning

What is reasoning?

Using facts and logic to arrive at an answer.

Deductive reasoning (or deduction): Making an inference based on widely accepted facts or premises.

General facts → Result

Inductive reasoning (or induction): Making an inference based on an observation (often an observation of a sample).

Observation → Generalization

Abductive reasoning (or abduction): Making a probable conclusion from what you know.

Known → Probable conclusion

From: Merriam-Webster

What type of reasoning is this?

You have a 10 o’clock appointment with the dentist.

You know it takes 30 minutes to go from your house to the dentist’s.

From the two facts, you [?] that you will have to leave your house at 9:30, at the latest, to be at the dentist’ on time.

A. deduct
B. induce
C. abduct

From: Merriam-Webster

What type of reasoning is this?

At lunch you observe 4 of your 6 coworkers ordering the same sandwich.

You [?] that the sandwich tastes good and worth trying it yourself.

A. deduct
B. induce
C. abduct

From: Merriam-Webster

What type of reasoning is this?

On a kitchen counter, there is a half-eaten sandwich.

You [?] that your teenage son made the sandwich and then saw that he was late for work. In a rush, he put the sandwich on the counter and left.

A. deduct
B. induce
C. abduct

A note of morphology

All “deduction”, “induction” and “abduction” are based on Latin ducere, meaning “to lead”.

The prefix **de-** means “from”
Deduction derives from generally accepted statements or facts.

The prefix **in-** means “to” or “toward”
Induction leads you to a generalization.

The prefix **ab-** means “away”.
You take away the best explanation in abduction.

From: Merriam-Webster

Formal vs informal reasoning

Formal reasoning: Follows formal rules of logic along with axiomatic knowledge to derive conclusions.

Informal reasoning: Uses intuition, experience, common sense to arrive at answers.

For most of this lecture, by “reasoning” we mean informal deductive reasoning, often involving multiple steps.

Technologies for improving reasoning

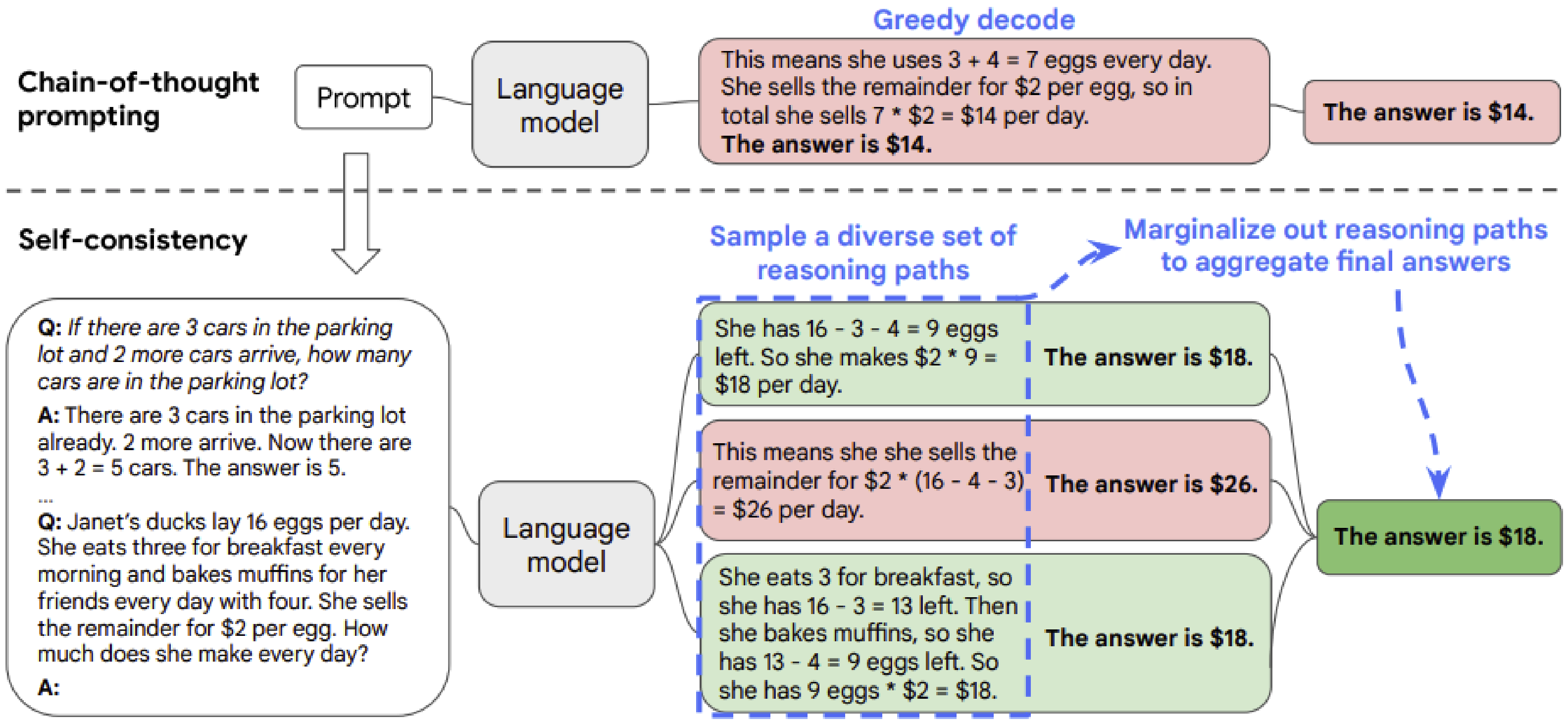
Recent popular technologies that improve LLM’s reasoning capabilities:

- Self-consistency.
- Tree of thought.
- Special tokens.

Self-consistency

For a problem:

- Generate multiple chain-of-thought (reasoning paths)
- Do a majority voting. Choose the most consistent answer.

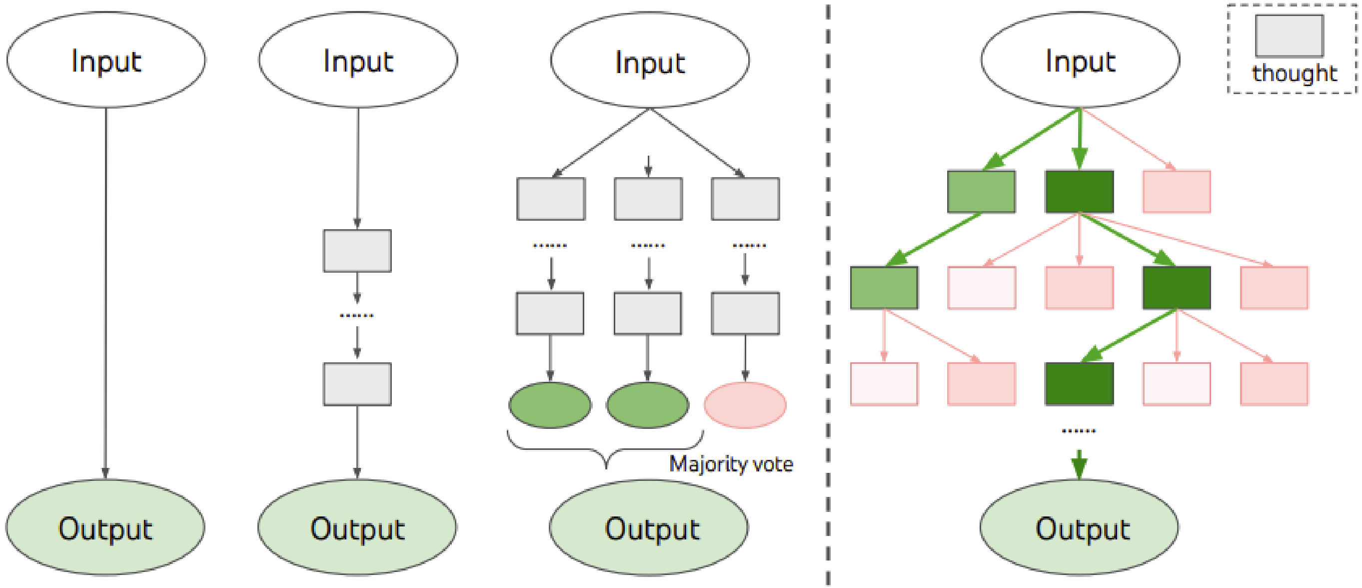


[Self-consistency improves chain-of-thought reasoning in language models. Yao et al., (2023)]

Tree-of-thought

Incorporate searching steps in the "thought".
Here's a CoT example:

"Let me try method A.
Method A leads to this immediate result.
The next step doesn't seem to go anywhere.
Let me try method B.
Method B leads to result R.
This seems possible.
The answer is R."



(a) Input-Output Prompting (IO)
(c) Chain of Thought Prompting (CoT)
(c) Self Consistency with CoT (CoT-SC)
(d) Tree of Thoughts (ToT)

[Deliberate problem solving with large language models. Yao et al., (2023)]

Leveraging special tokens for reasoning

The reasoning process can be strengthened via special tokens.
The model need to be trained to use these special tokens.
Here’s an example using the CoT in the previous slide:

Let me try method A.
Method A leads to this immediate result.
The next step doesn’t seem to go anywhere.
Let me try method B.
Method B leads to result R.
This seems possible.
The answer is R.

<try>Let me try method A.
Method A leads to this immediate result.
<eval>The next step doesn’t seem to go anywhere.
<backtrace>
<try>Let me try method B.
Method B leads to result R.
<eval> This seems possible.
<report>The answer is R.

Leveraging special tokens for tool usage

These special tokens don’t need to be just about the reasoning.
Here’s an example CoT about tool usage.

Now I’m going to compute 23456*34567. I’m going to write a Python script to help me solve it.
<code>23456 * 34567</code>
<execute>

Send the code to sandbox [1]

Result is 810803552

Now I’m going to compute 23456*34567. I’m going to write a Python script to help me solve it.
<code>23456 * 34567</code>
<execute>
The result is 810803552.

[1] In Deep Agents, sandboxes are backends that define the environment where the agent operates

From reasoning to explanation

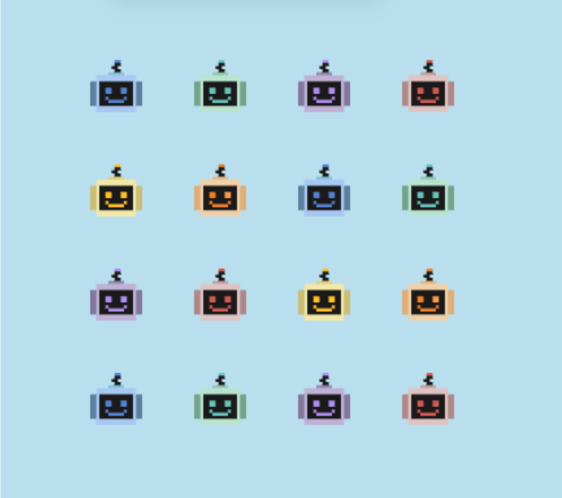
Explanation is approximately “reasoning + audience”.

- Reasoning only considers the correctness.
- Explanation also considers the human side.

- Reasoning refers to the process (“reason” is the product).
- Explanation can refer to both the process and the product.

Recap for reasoning

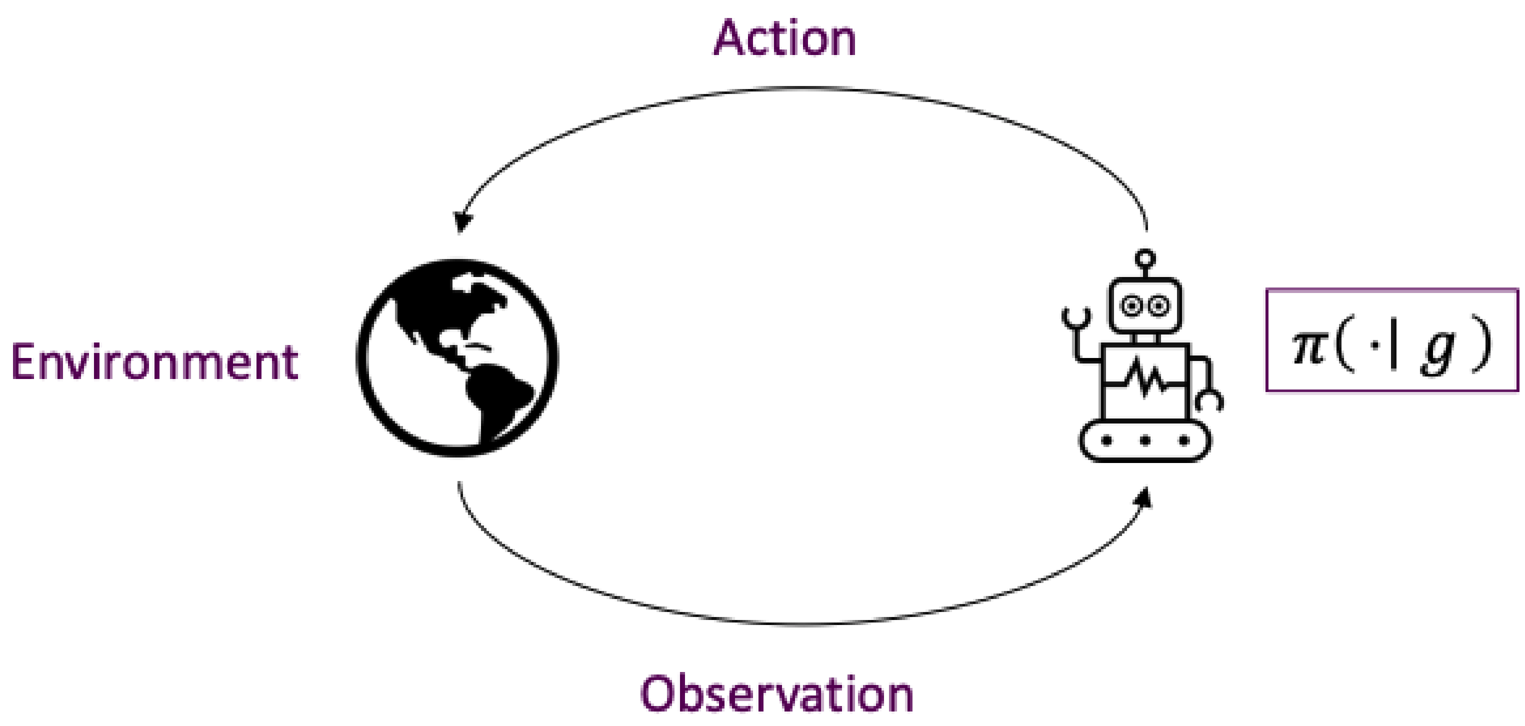
- Three types of reasoning: deductive vs inductive vs abductive
- Prompting methods for enhancing reasoning: self-consistency, tree-of-thought, special tokens.



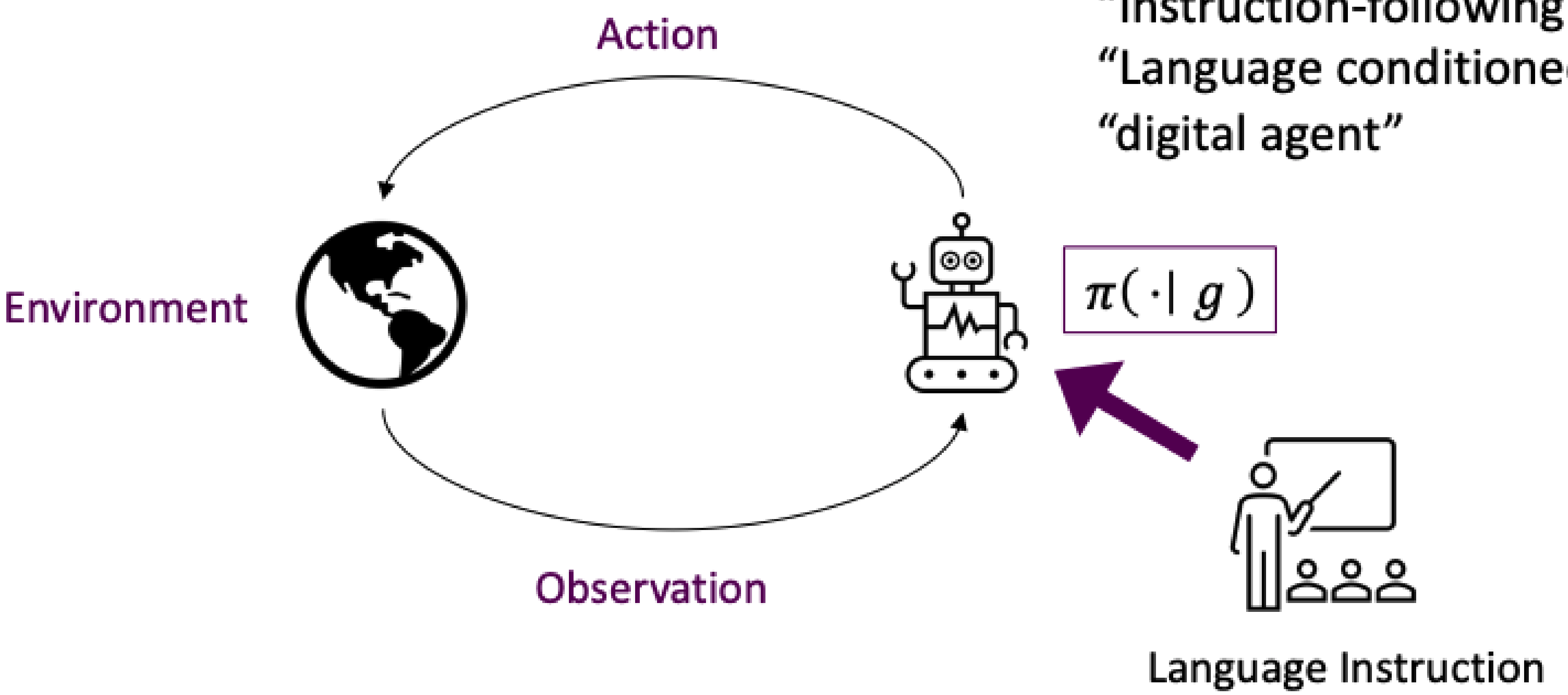
Language Model Agents

Image source: AG2

Some terminology



Some terminology

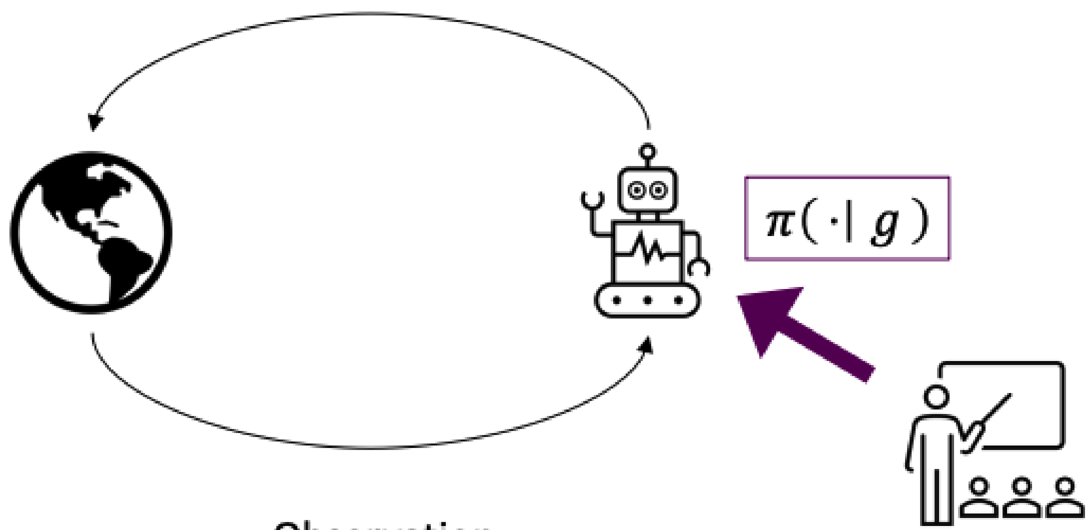


"Instruction-following agent"
"Language conditioned policy"
"digital agent"

Some terminology

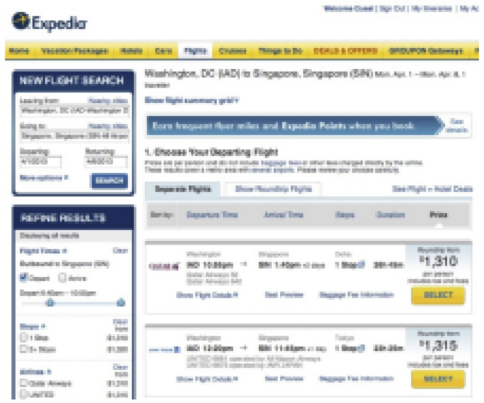
`Type … on …, Click on …, Choose … from dropdown, …`

Environment

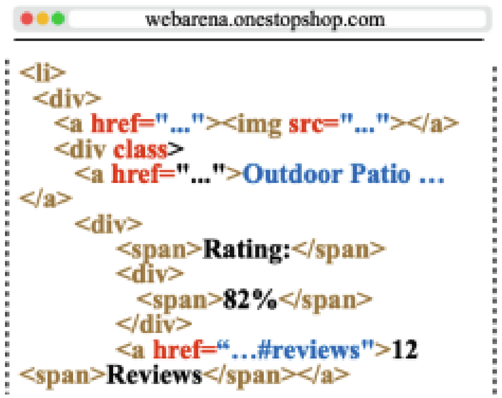


Observation

"Book a flight from San Francisco to New York"



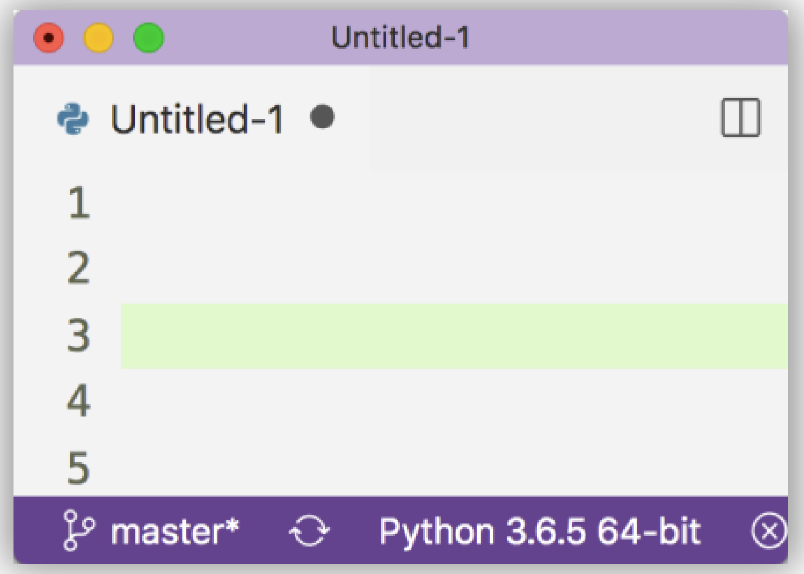
Raw pixels as observation?



```
webarena.onestopshop.com
<li>
 <div>
   <a href="..."><img src="..."></a>
   <div class>
     <a href="...">Outdoor Patio ...
</a>
     <div>
          <span>Rating:</span>
          <div>
            <span>82%</span>
          </div>
          <a href="...#reviews">12
<span>Reviews</span></a>
```

HTML DOM as observation?

Applications: Natural Language Interfaces



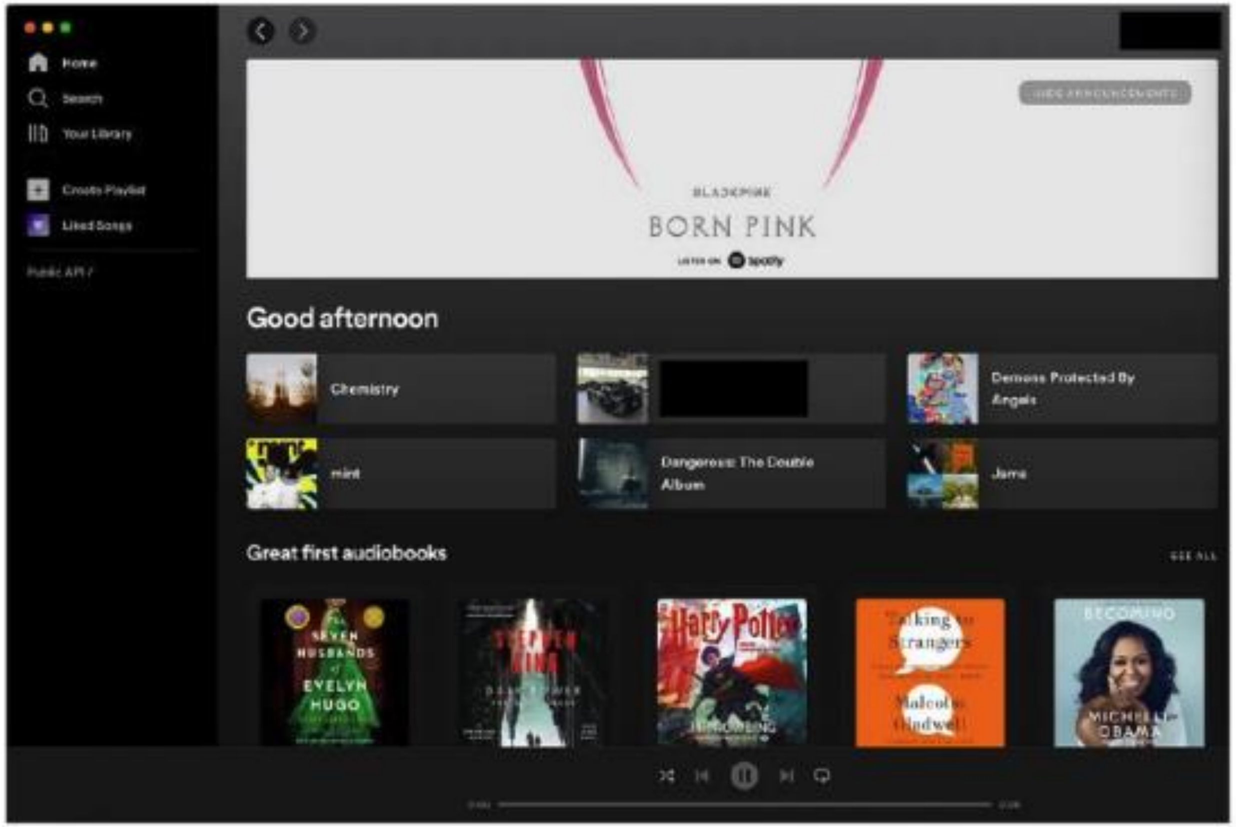
Virtual Assistants

- *Set an alarm at 7 AM*
- *Remind me for the meeting at 5pm*
- *Play Jay Chou's latest album*

Natural Language Programming

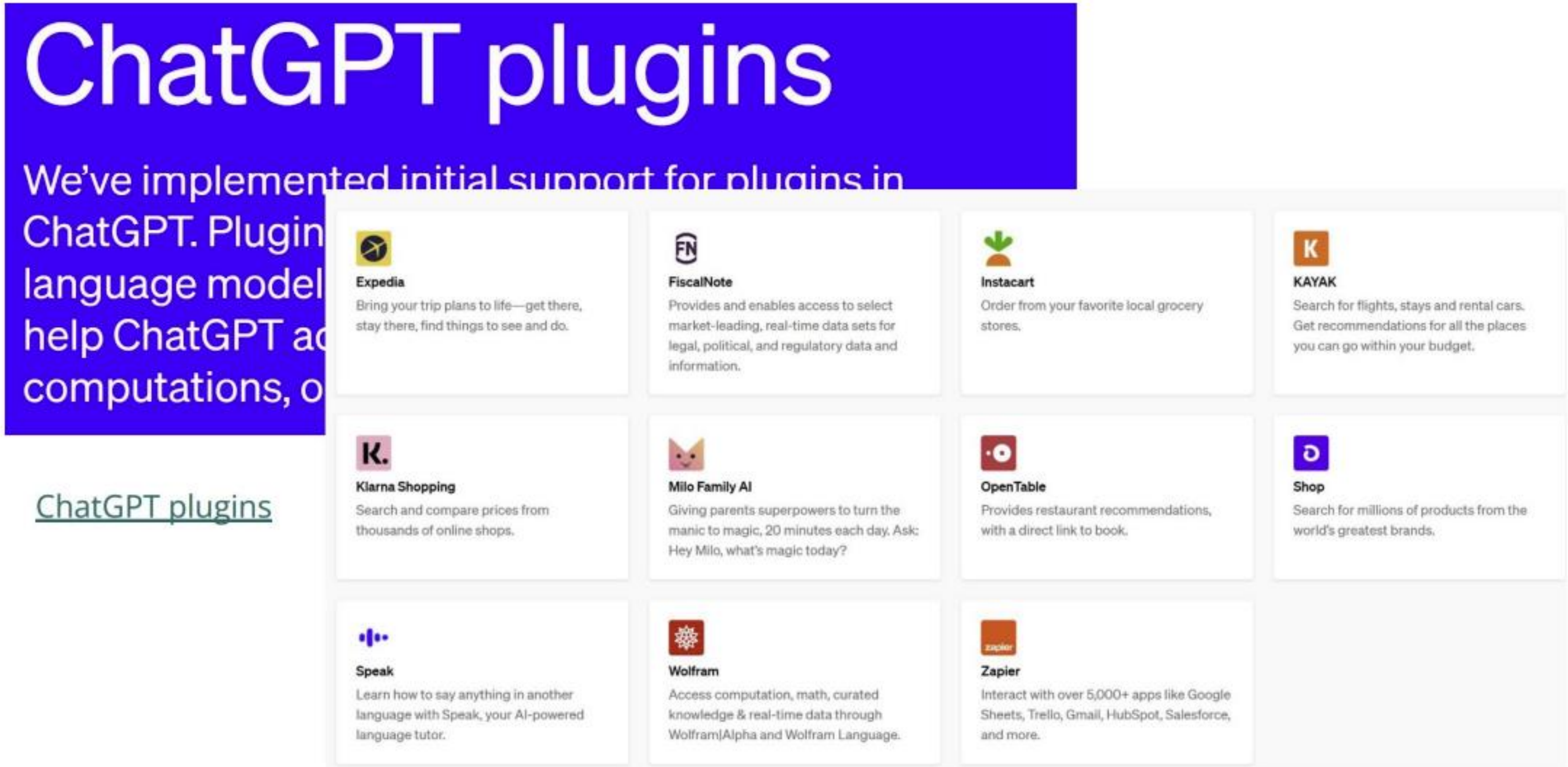
- *Sort my_list in descending order*
- *Copy my_file to home folder*
- *Dump my_dict as a csv file output.csv*

Applications: UI automation



"Play some synthwave songs"

Applications: Multi-step “Tool use”



ChatGPT plugins

A simple LM Agent with ReACT

You are an agent capable of the following actions:

1. Type X on Y
2. Move mouse to
3. Click on X
4. Type Char x on Y

Your objective is to follow user instructions, by mapping them into a sequence of actions. Instruction: {g}

So far, you have taken the following actions and observed the following environment states:
Previous Actions and Observations:
o1: … a1: …
o2: … a2: …

After executing these actions, you observe the following HTML state: <HTML state>
Now, think about your next action: Thought: [model-pred]
Now, take an action: Action: [model-pred]

Action space
Instruction
Previous observations and actions
Provide current observation in text

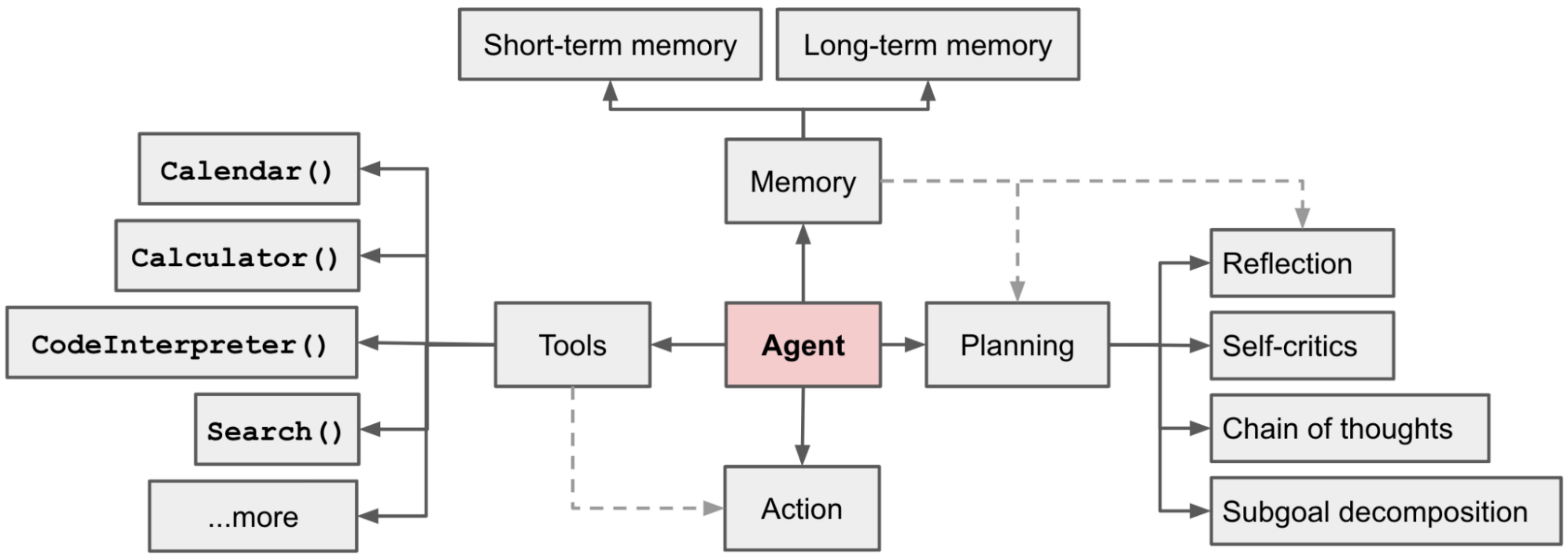
LM generates next action. Use that action to update environment and repeat!

Mostly just CoT prompting in a loop.

ReAct: Synergizing Reasoning and Acting in LMs. Yao et al., 2022

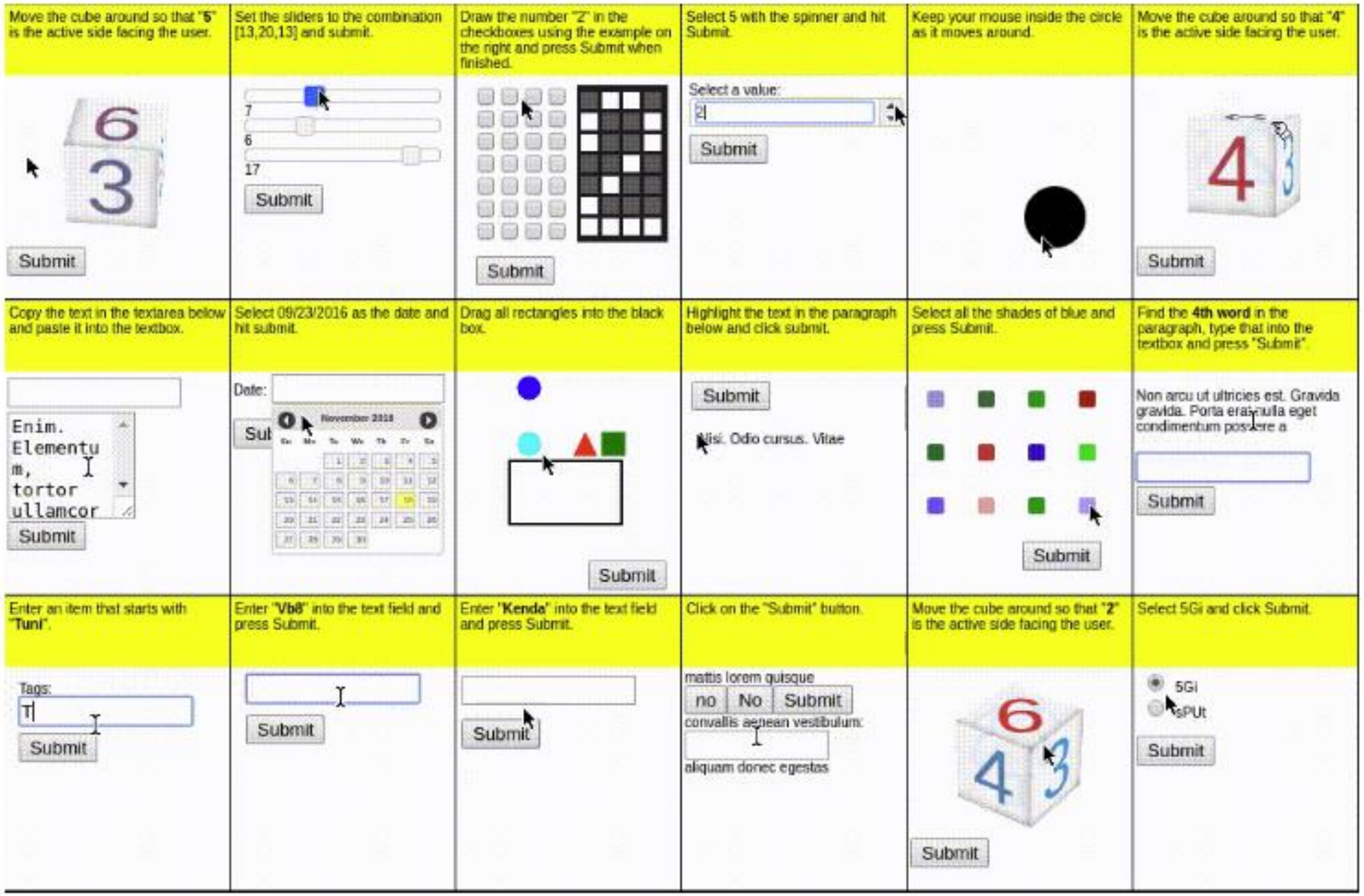
Some popular components for LM Agents

Memory. Tool-using. Planning. Action.



LLM-powered Autonomous Agents. Lilian Weng Blog 2023

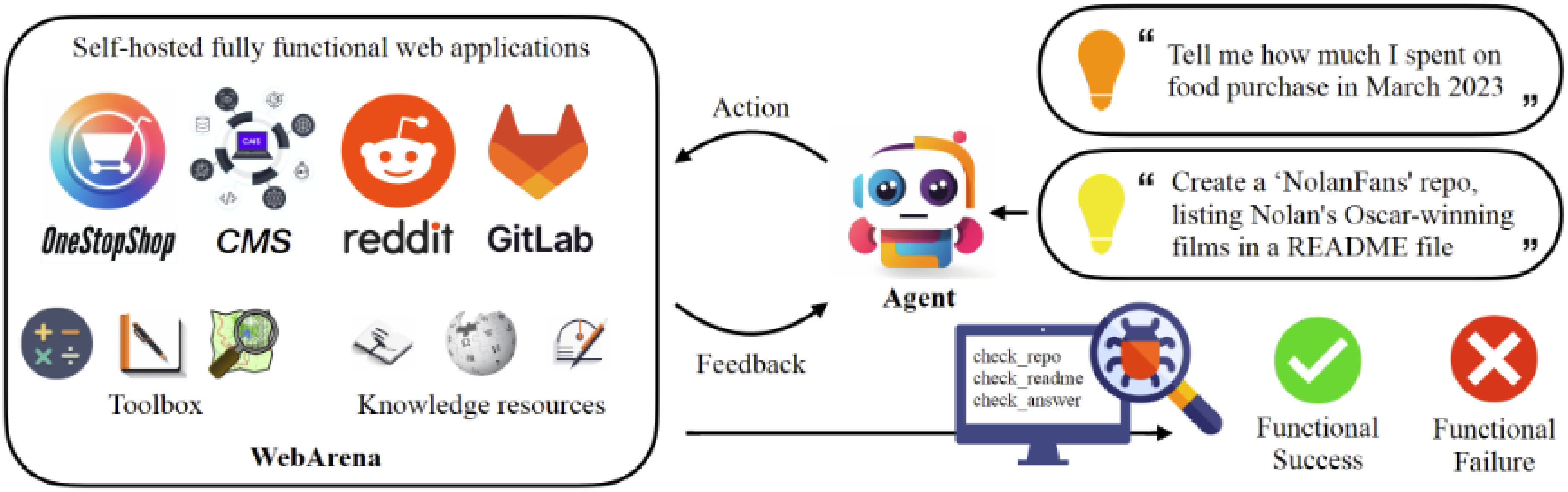
A benchmark for Agents: MiniWob++



- Sandboxed environment evaluating basic browser interactions across a range of applications from social media to email clients
- Evaluates functional correctness
- Not real world (limited functionality)
- Relatively short-horizon
- Zero-shot performance far from perfect!

https://miniwob.farama.org/

A benchmark for Agents: WebArena



WebArena is a standalone, self-hostable web environment for building autonomous agents. WebArena creates websites from four popular categories with functionality and data mimicking their real-world equivalents. To emulate human problem-solving, WebArena also embeds tools and knowledge resources as independent websites. WebArena introduces a benchmark on interpreting high-level realistic natural language command to concrete web-based interactions. We provide annotated programs designed to programmatically validate the functional correctness of each task.

https://webarena.dev/

Let’s define the formats for tool-calling

- It’s possible to use many formats: JSON, Markdown, XML, etc., when connecting the agents and data sources and external systems.
  - It’s be great to have a unified protocol.
- A communication protocol is proposed by Anthropic: Model Context Protocol (MCP)
- MCP defines many types, for example:
  - JSONRPCRequest
  - ClientCapabilities
  - AudioContent
  - …
- Based on MCP (the protocol), numerous MCP servers are established
  - They function like the “USB type-C” interface.

Putting relevant contents into the prompt

- There are multiple entries in a database.
  - Each entry can be a document.
- Only one, or a few entries are relevant at a time.
- So, search for the relevant entries
  - Using e.g., max-similarity-search
- And then, put only the relevant entries in the prompt.
- This framework is referred to as Retrieval-Augmented Generation (RAG).

Making the system prompts more informative

- Before answering the user prompts, modern LLM providers prepend a system prompt.
- A default system prompt is: You are a helpful assistant.
- System prompts can be extended. Several examples:
  - In OpenClaw, the system prompts include a file, soul.md (among many others)

    soul.md describes the persona and expected response styles of the agent.
  - In Claude Code, the system prompts include files, CLAUDE.md and MEMORY.md (among many others)

    CLAUDE.md overviews the repository's contents and architecture.

    MEMORY.md functions as a "scratchpad" for the agent.

    They are usually placed under the ~/.claude directory.
- There are also other files under the ~/.claude directory, for example, skill files.

Extending the agent with skills:

- Here's an example, from the Claude Code doc
  - A markdown file is needed at: ~/.claude/skills/explain-code/SKILL.md
  - The name becomes the Claude's slash command. This skill can be invoked with /explain-code filename

```
---
name: explain-code
description: Explains code with visual diagrams and analogies. Use when explai
---

When explaining code, always include:

1. **Start with an analogy**: Compare the code to something from everyday life
2. **Draw a diagram**: Use ASCII art to show the flow, structure, or relations
3. **Walk through the code**: Explain step-by-step what happens
4. **Highlight a gotcha**: What's a common mistake or misconception?

Keep explanations conversational. For complex concepts, use multiple analogies
```

Practical considerations and safety concerns

How many memory do I need to run the LLM?

To run inference:

- We'll put all parameters onto the GPU, plus the context/etc.
- X billion parameters need at least 4X GB memory on GPU on "full precision"
  - i.e., with float32, or fp32, or torch.bfloat32, or equivalent 32-bit data type.
- Remember that 8 bit is one byte
- Most models on Huggingface specify the size in their names.

- On half precision (16-bit): XB parameter ~ 2X GB memory.
- Further quantization techniques can further compress the models to 8-bit or 4-bit level.

Exercise: I have one NVidia RTX 4090 card (24GB memory), to run inference on the following model, what precision can I run up to?

1. meta-llama/Llama 3.1-8B-Instruct
2. google/gemma-2-2b-it
3. Qwen/QwQ-32B-Preview

How many memory do I need to run the LLM?

To run (full-parameter) training:

- Depending on the optimizer, spaces equivalent to 2-4 copies of the full parameters will be needed on GPU memory.
- SGD: 2 copies
  - New and old params.
- Adam: 4 copies
  - New param, old param, and two pieces of momentum parameters.

Exercise: Which GPU can train an 8B model using SGD (assume half precision)? How about Adam optimizer?

1. 4090: 24GB
2. A6000: 48GB
3. H100: 80GB

The LLM struggle at long contexts

"Lost in the middle"

ACL Anthology
https://aclanthology.org › 2024.tacl-1.9

Lost in the Middle: How Language Models Use Long ...

by NF Liu · 2024 · Cited by 924 — Abstract. While recent language models have the ability to take long contexts as input, relatively little is known about how well they use longer context.

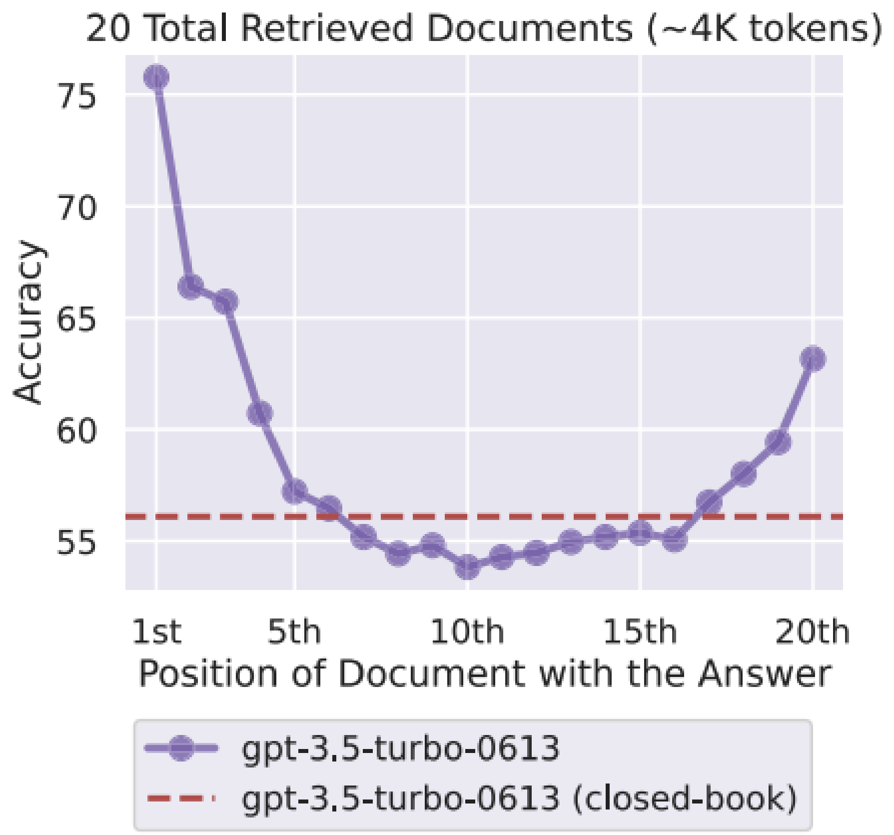


Figure 1: Changing the location of relevant information (in this case, the position of the passage that answers an input question) within the language model's input context results in a U-shaped performance curve—models are better at using relevant information that occurs at the very beginning (primacy bias) or end of its input context (recency bias), and performance degrades significantly when models must access and use information located in the middle of its input context.

Factuality and Hallucination

- Language models model distributions over text, not facts. There's no guarantee that what they generate is factual:
  - Language models are trained on the web. Widely-popularized falsehoods may be reproduced in language models.
  - A language model may not be able to store all rare facts, and as a result moderate probability (uncertainty or lack of strong evidence) is assigned to several options.
- There are many proposed solutions to factuality. How do we evaluate them? How do we check facts "explicitly"?

Hallucination

LLMs are prone to generate untruthful information that either conflicts with the existing source or cannot be verified by the available source. Even the most powerful LLMs such as ChatGPT face great challenges in migrating the hallucinations the generated texts. This issue can be partially alleviated by special approaches such as alignment tuning and tool utilization.

Factuality and Hallucination

There are still many limitations of large LMs (size, hallucination) that may not be solvable with RLHF!

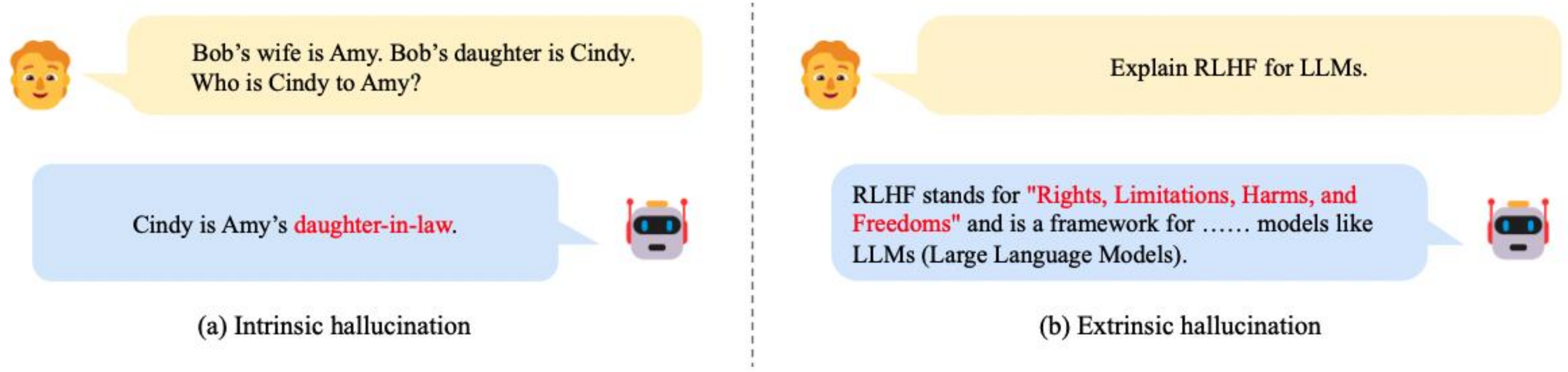


Fig. 14: Examples of intrinsic and extrinsic hallucination for a public LLM (access date: March 19, 2023). As an example of intrinsic hallucination, the LLM gives a conflicting judgment about the relationship between Cindy and Amy, which contradicts the input. For extrinsic hallucination, in this example, the LLM seems to have an incorrect understanding of the meaning of RLHF (reinforcement learning from human feedback), though it can correctly understand the meaning of LLMs (in this context).

Knowledge recency

- LLMs would encounter difficulties when solving tasks that require the latest knowledge beyond the training data.
- To tackle this issue, a straightforward approach is to regularly update LLMs with new data.
- However, it is very costly to fine-tune LLMs, and also likely to cause the catastrophic forgetting issue when incrementally training LLMs.
- Therefore, it is necessary to develop efficient and effective approaches that can integrate new knowledge into existing LLMs, making them up-to-date.

> **Knowledge Recency**
>
> The parametric knowledge of LLMs is hard to be updated in a timely manner. Augmenting LLMs with external knowledge sources is a practical approach to tackling the issue. However, how to effectively update knowledge within LLMs remains an open research problem.

Privacy concern

Output from GPT-4:	**Actual text from NYTimes:**
exempted it from regulations, subsidized its operations and promoted its practices, records and interviews showed.	exempted it from regulations, subsidized its operations and promoted its practices, records and interviews showed.
Their actions turned one of the best-known symbols of New York — its yellow cabs — into a financial trap for thousands of immigrant drivers. More than 950 have filed for bankruptcy, according to a Times analysis of court records, and many more struggle to stay afloat.	Their actions turned one of the best-known symbols of New York — its signature yellow cabs — into a financial trap for thousands of immigrant drivers. More than 950 have filed for bankruptcy, according to a Times analysis of court records, and many more struggle to stay afloat.
"Nobody wanted to upset the industry," said David Klahr, who from 2007 to 2016 held several management posts at the Taxi and Limousine Commission, the city agency that oversees medallions. "Nobody wanted to kill the golden goose."	"Nobody wanted to upset the industry," said David Klahr, who from 2007 to 2016 held several management posts at the Taxi and Limousine Commission, the city agency that oversees cabs. "Nobody wanted to kill the golden goose."
New York City in particular failed the taxi industry, The Times found. Two former mayors, Rudolph W. Giuliani and Michael R. Bloomberg, placed political allies inside the Taxi and Limousine Commission and directed it to sell medallions to help them balance budgets and fund key initiatives.	New York City in particular failed the taxi industry, The Times found. Two former mayors, Rudolph W. Giuliani and Michael R. Bloomberg, placed political allies inside the Taxi and Limousine Commission and directed it to sell medallions to help them balance budgets and fund priorities. Mayor Bill de Blasio continued the policies.
	Under Mr. Bloomberg and Mr. de Blasio, the city made more than $855 million by selling taxi medallions and collecting taxes on private sales, according to the city.
During that period, much like in the mortgage lending crisis, a group of industry leaders enriched themselves by artificially inflating medallion prices. They encouraged medallion buyers to borrow as much as possible and ensnared them in interest-only loans and other one-sided deals that often required borrowers to pay hefty fees, forfeit their legal rights and give up most of their monthly incomes.	But during that period, much like in the mortgage lending crisis, a group of industry leaders enriched themselves by artificially inflating medallion prices. They encouraged medallion buyers to borrow as much as possible and ensnared them in interest-only loans and other one-sided deals that often required them to pay hefty fees, forfeit their legal rights and give up most of their monthly incomes.
When the market collapsed, the government largely abandoned the drivers who bore the brunt of the crisis. Officials did not bail out borrowers or persuade banks to soften loan	

Exhibit J of NY Times vs OpenAI (2023), tweeted by: https://x.com/CeciliaZin/status/1740109465918328844

Interpretability

- The ability to understand and explain the decisions or predictions of NLP models.
- Ensuring model trustworthiness, identifying biases, and providing insights into model behavior.
  - Feature Importance: which features are most important for making predictions
  - Local interpretability: explain model predictions on specific instances
  - Global Interpretability: This involves understanding the overall behavior of the model
  - Attention Mechanisms: provide insights into which parts of the input are being attended to during processing.
  - Word Embedding Analysis: provide insights into the relationships between words in the model's learned space; t-SNE or PCA for visualization.
  - Visualizations: help in understanding its decision-making process. For example, heatmaps can be used to highlight important words or phrases.
  - Bias Detection and Mitigation: Interpretability methods can be used to identify and mitigate biases in NLP models, helping to ensure fairness and ethical use.
  - Error Analysis: Examining cases where the model makes mistakes can provide insights into its limitations and areas for improvement.
  - And many others ...

Ethics

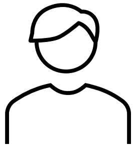
Here are some of the key concerns of ethical issues associated with NLP models.

- Bias and Fairness: learn biases present in the data they are trained on, leading to unfair or discriminatory outcomes.
- Transparency and Explainability: it is challenging to explain how and why a model makes certain predictions, which is crucial for trust and accountability.
- Data Privacy and Security: NLP models often require large amounts of data to be trained effectively. If not handled properly, sensitive information could be inadvertently disclosed.
- Misinformation and Disinformation: NLP models can be used to generate fake news or deceptive content. This poses a significant challenge for combating misinformation and disinformation online.
- Environmental Impact: Large NLP models often require significant computational resources to train and deploy. This can have a substantial environmental impact due to the energy consumption associated with high-performance computing.
- And many more …

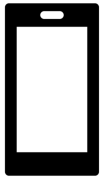
A case study: adversarial helpfulness

Post-hoc black-box explanations are convincing.

Here’s a problem. Explain it to me.
What does the government sometimes have too much of?
A. Canada. B. Trouble. C. City. *D. Control*. E. Water



"Control" is a common concern related to governments, as too much control can imply excessive regulation, loss of personal freedoms, or authoritarian practices.



[LLM-Generated Black-Box Explanations Can Be Adversarially Helpful. Ajwani et al., (2024)]

A case study: adversarial helpfulness

But the explanation towards incorrect problems can also be highly convincing.

Here's a problem.
What does the government sometimes have too much of?
A. Canada. B. Trouble. C. City. *D. Control*. E. Water
Explain to me why B is the correct answer.

Here's why B. Trouble is the correct answer.
[...]
Elimination of Other Options:
[...] D. Control: While some might argue that certain governments have too much control, this is more of a subjective opinion rather than a generally accepted issue.
[...]
Logical Fit: "Trouble" fits logically as governments are often described as having many problems or facing numerous difficulties, making it the most reasonable choice among the given options.

LLMs frequently *reframe the problems, demonstrate elevated confidence, and present selective facts*, when explaining even the wrong problems.

LLM-Generated Black-Box Explanations Can Be Adversarially Helpful. Ajwani et al., (2024)

Summary

Reasoning

Language Model Agent

Practical considerations and safety concerns

- Memory consumption
- Long context
- Hallucination, knowledge recency
- Privacy
- Interpretability and ethics
- Many more

Readings

1. Language Models are Few-Shot Learners
2. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
3. Finetuned Language Models Are Zero-Shot Learners
4. Learning to summarize from human feedback

Acknowledgements:

The slides are developed from Stanford's CS224n and CMU's Advanced NLP, and Stevens' CS 584 NLP lecture slides by Zining Zhu, Ping Wang, and Yue Ning.